

universidad de León Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

Ingeniería del software seguro en la era de la IA generativa

Diseño y Programación Seguras

Prácticas de laboratorio

Francisco Javier Rodríguez Lera

Material elaborado con asistencia de IA generativa (DeepSeek-V4-Flash; considerados también GPT-6 y Claude Sonnet 4.6). Contenido revisado y validado por el autor.



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías

Industrial, Informática y Aeroespacial

universidad de León

Objetivos de aprendizaje

Al terminar el anexo, el estudiante será capaz de:

- **Localizar** y reproducir un fallo funcional pero inseguro (práctica 01).
- **Comparar** el efecto del *prompt* frente a la especificación (práctica 02).
- **Construir** un *pipeline* aislado con puertas *fail-closed* (práctica 03).
- **Verificar** un componente de suma comprobada con contratos y pruebas.
- **Trabajar** siempre en aislamiento y con datos sintéticos.

Marco común y condiciones de aislamiento

- **Aislamiento:** las operaciones que ejecutan código no fiable se realizan en VM o aislamiento equivalente comprobado (usuario no privilegiado, directorio efímero, límites, sin red). Un contenedor convencional no es, por sí solo, frontera suficiente.
- **Sin servicios comerciales obligatorios:** herramientas de código abierto y datos sintéticos (compilador C, Python 3, `pytest`, Bandit, `pip-audit`, VM o `bubblewrap`).
- **Sin datos reales sensibles:** solo CSV sintéticos con valores ficticios.
- **Agentes con permisos limitados:** sin red, sin escritura fuera del directorio de trabajo, sin secretos, aprobación humana para acciones sensibles.

Registrar la **tupla** (modelo, versión, configuración, *prompt*/contexto, herramientas, entorno) *antes* de generar o ejecutar es parte de la reproducibilidad.

Trazabilidad prácticas ↔ objetivos

Objetivo de aprendizaje	P. 01	P. 02	P. 03
Distinguir código funcional de correcto y seguro	●	●	
Formular precondiciones, postcondiciones e invariantes		●	
Identificar familias de vulnerabilidad por CWE	●	●	
Explicar por qué las pruebas finitas no demuestran seguridad	●	●	●
Diseñar un protocolo reproducible (tupla)		●	
Diferenciar sistema de IA regulado de software convencional		●*	
Distinguir sandbox regulatorio, aislamiento técnico y pipeline			●
Modelar amenazas de agentes y límites de confianza		●	●
Diseñar un pipeline con puertas y trazabilidad			●
Aplicar el AI-SDLC y reconocer sus límites		●	●

●* indica análisis documental breve, sin simular participación en un sandbox regulatorio. **Propuesta docente**.

Práctica 01: código funcional pero vulnerable

- Dos artefactos que *funcionan* con entradas ordinarias pero no son seguros:
  - Comprobación de ruta antes de usarla, con ventana *TOCTOU* (CWE-367) que puede involucrar enlaces simbólicos (CWE-59).
  - Exportación de celdas que no neutraliza fórmulas de hoja de cálculo (CWE-1236).
- El estudiante debe localizar el fallo, reproducirlo *solo en aislamiento*, proponer una corrección contextual y explicar qué *no* cubre esa corrección.

La aplicabilidad de CWE-59 depende de la implementación concreta. La clasificación CWE se refiere al mecanismo de fallo, no a una garantía de explotación universal.

Práctica 01 - resumen

Apartado	Contenido
Objetivo	Distinguir funcional de correcto/seguro; localizar un fallo de E/S y clasificarlo por CWE; reproducirlo en aislamiento; proponer corrección contextual y sus límites
Se entrega	Código fuente de los dos artefactos y las dos versiones corregidas; registro de comandos y salidas; clasificación CWE justificada
Evidencias	Pruebas positiva y negativa de cada artefacto; nota de trazabilidad fallo $\rightarrow$ requisito incumplido $\rightarrow$ control que lo neutraliza
Rúbrica	Localización (20 %); CWE (15 %); explotación en aislamiento (25 %); corrección (20 %); trazabilidad (10 %); reflexión (10 %)

La prueba negativa del TOCTOU es *no determinista*: observarlo no demuestra que sea inevitable, y no observarlo no demuestra que sea seguro.

¿Por qué pasar las pruebas no basta?

- ¿Por qué un programa que pasa todas las pruebas funcionales puede seguir siendo inseguro?
- Dad un ejemplo tomado de la práctica 01.

Práctica 02: prompt incompleto frente a especificación

- Se plantea el *mismo* problema dos veces: con un *prompt* mínimo (sin requisitos de seguridad) y con una especificación completa (requisitos F1-F4, de seguridad S1-S6, restricciones R1-R3).
- Los dos artefactos se auditan con la misma lista de criterios y se comparan.
- Objetivo: poner a prueba la hipótesis rectora: que la IA desplaza el esfuerzo hacia la especificación, la revisión y la validación, sin eliminar la responsabilidad sobre el resultado.

La comparación mide *qué se ha cubierto*, no *qué riesgo queda*. Completar la especificación puede mejorar el artefacto, pero ese efecto debe medirse y no se presupone.

Práctica 02 - resumen

Apartado	Contenido
Objetivo	Comprender el desplazamiento del esfuerzo; formular especificación funcional y de seguridad; auditar dos artefactos con los mismos criterios y clasificar por CWE
Se entrega	Los dos artefactos sin modificar; registro de condiciones de generación; tabla de auditoría S1-S6 por artefacto; tabla de comparación y conclusión
Evidencias	Clasificación CWE de cada hallazgo con línea afectada; transcripción de las pruebas P1-P5 sobre ambos artefactos
Rúbrica	Registro de condiciones (15 %); auditoría S1-S6 (25 %); CWE (15 %); comparación A/B (20 %); pruebas (15 %); conclusión (10 %)

No modificar el artefacto antes de auditarlo: corregir antes de medir destruye la comparación.

Práctica 03: pipeline aislado de validación

- Construye un *pipeline* que convierte un artefacto candidato en un expediente con evidencia verificable: SAST, SCA, compilación/importación, pruebas funcionales y adversarias con `pytest`, recogida de evidencias.
- **Toda etapa que ejecute código no fiable** (incluida la importación y las pruebas) se ejecuta *dentro* del aislamiento, no tras una comprobación aislada que solo certifique el sandbox.
- Cada etapa es una *puerta fail-closed*: ante duda, fallo de herramienta, error de infraestructura o evidencia ausente, se cierra.
- Los hallazgos críticos se corrigen, se demuestran falsos positivos o se tratan con una excepción formalmente autorizada, delimitada y registrada.

Práctica 03 - resumen

Apartado	Contenido
Objetivo	Diseñar y ejecutar un *pipeline* que produzca un expediente con evidencia; puertas *fail-closed*; trazabilidad requisito→implementación→prueba→evidencia; reconocer límites
Se entrega	Script de orquestación y envoltorio de *sandbox*; `spec.md`; informe SAST con triaje y SCA; salida de `pytest`; registro de la ejecución aislada
Evidencias	Matriz de trazabilidad auditable con oráculos que inspeccionan propiedades (no solo existencia de archivos); prueba de que el pipeline rechaza un artefacto defectuoso; límites declarados
Rúbrica	Diseño (20 %); puertas *fail-closed* (20 %); SAST/SCA/triaje (15 %); pruebas y *sandbox* (20 %); trazabilidad (15 %); límites (10 %)

Un pipeline verde no demuestra seguridad: es una batería finita sobre un conjunto declarado de propiedades.

Comprobación negativa de las puertas

- La práctica 03 no termina cuando todo está en verde: debe incluir al menos una **prueba de fallo deliberado**.
- Ejemplo: introducir en una copia controlada una exportación XLSX que escriba como fórmula un valor CSV que empiece por =. El oráculo inspecciona el *tipo y contenido* de la celda, no solo que el archivo existe.
- El pipeline debe devolver un estado de **rechazo** y conservar la evidencia de ese rechazo.
- Probar también un fallo de infraestructura y la ausencia de un informe obligatorio: ambos deben bloquear la aceptación.
- No se ejecutan muestras maliciosas reales fuera del entorno autorizado.

Práctica 04 (opcional): el kit AI-SDLC sobre el caso

- Aplicar `P00_integral` y el runner de `kit_agentes_ai_sdlc_v2` al caso CSV→PDF/XLSX, registrar evidencias y decidir con revisión humana.
- **Preparación:** Python 3.10+; `bubblewrap` si se usa `-sandbox`; herramientas opcionales (`ruff`, `mypy`, `bandit`, `pip-audit`) para `extended`.
- Fijar criterios y pruebas de aceptación *antes* de generar; ejecutar con perfil básico y aislamiento; registrar `SKIP` y `NOT_EVALUATED` como limitaciones, no como éxitos.
- Completar la evidencia (requisito, comprobación, herramienta y versión, comando, resultado, log, limitación); decidir con una persona de rol distinto del generador.
- Si falta evidencia, la decisión es *no aceptar*.

El runner *ejecuta código*: no lo uses sobre código no fiable fuera de un sandbox. Un pipeline verde no demuestra seguridad, corrección ni conformidad.

Práctica técnica: verificación de suma comprobada

- Aplica conjuntamente contratos, invariantes, pruebas de propiedades, *fuzzing* y verificación formal sobre un componente mínimo: `checked_add_u32` (suma de enteros sin signo de 32 bits).
- Contrato: `result` apunta a objeto válido; si $a + b$ es representable, devuelve `true` y escribe el valor; si excede `UINT32_MAX`, devuelve `false` y *no modifica* `*result`.
- Distinción clave: la aritmética sin signo en C es *modular* y definida (sin comportamiento indefinido al desbordar), pero el contrato exige el *valor matemático* o el *rechazo*.

Que `a + b` se evalúe sin comportamiento indefinido no significa que su resultado cumpla el contrato. La terminación es trivial (sin bucles), así que corrección total = parcial.

Resultados de verificación de las tres implementaciones

Criterio	Referencia	Candidata A	Candidata B
Compila con `-Werror`	Sí	Sí	Sí
Contrato (frontera y preservación)	Pasa	Falla	Falla
Propiedades P1-P5	Pasa	Viola P1	Viola P3
ASan / UBSan	Sin hallazgos	Sin hallazgos	Sin hallazgos
Fuzzing (P1-P4)	Sin fallos	Detecta	Detecta

Las tres *compilan sin advertencias* y pasan los sanitizers, pero dos incumplen el contrato: solo las pruebas que ejercen las cláusulas lo revelan. «Compila» y «pasa sanitizers» $\neq$ «correcto».

Propiedades, fuzzing y análisis

- Propiedades: P1 (resultado representable), P2 (detección del desbordamiento), P3 (preservación del estado en error), P4 (conmutatividad), P5 (determinismo).
- Generación determinista con semilla fija y 200 000 casos por propiedad: es una *muestra*, no una prueba exhaustiva (no agota las 2^{64} parejas).
- El *fuzzing* añade un harness que comprueba P1-P4; con la Candidata B se obtuvo un caso reproducible.
- Las propiedades P1-P5 e I1-I3 (acumulador) se demuestran formalmente con Frama-C/WP sobre variantes anotadas con ACSL: 94/94 obligaciones, sin pendientes.
- La demostración cubre el componente y su composición; *no* la concurrencia ni la validez general de punteros.

Modelo de amenazas y verificación composicional

- El modelo de amenazas distingue defectos funcionales y amenazas de seguridad: un rechazo indebido (Candidata A) es defecto de corrección; un desbordamiento no detectado podría derivar en una vulnerabilidad de memoria (CWE-190 hacia CWE-787).
- La clasificación depende del *contexto de uso*; no es automática.
- La verificación composicional se ilustra con un acumulador que usa `checked_add_u32`: su contrato depende del inferior (solo preserva su estado si el inferior cumple P3).
- Una variante que ignora el valor de retorno compila y pasa pruebas nominales, pero corrompe el estado: verificar cada componente por separado *no* basta.

Validez del oráculo y mutación

- Un oráculo defectuoso que usa una implementación candidata como referencia comparte el defecto: la prueba «pasa» aunque la implementación sea incorrecta.
- Un oráculo derivado de la especificación (suma en 64 bits) detecta el defecto.
- Tipos de oráculo: derivado de especificación, de implementación, metamórfico y comparación diferencial.
- Mutación: cambios controlados (estricta por no estricta, eliminar condición de error, escribir antes de comprobar, detección por acarreo). El mutante por acarreo es *equivalente* y no se detecta (se documenta).
- La puntuación de mutación estima la capacidad de detección; no garantiza corrección.

Trazabilidad de requisitos, propiedades y evidencias

Requisito	Prop.	Técnica	Evidencia	Limitación
Aceptar suma representable	P1	Pruebas, propiedades, fuzzing	200 000 casos + fuzzing	Muestra, no exhaustiva
Rechazar desbordamiento	P2	Pruebas de límites, propiedades	Frontera `UINT32_MAX`	Requiere caso dirigido
Preservar estado en error	P3	Pruebas, propiedades, fuzzing	Caso reproducible (B)	Ninguna para la referencia
Conmutatividad	P4	Propiedades, metamórfica	Pares de entradas	Solo entradas representables
Determinismo	P5	Propiedades	Repetición	Estado inicial equivalente
Corrección universal	P1-P5, I1-I3	Verificación formal (Frama-C/WP)	94/94 obligaciones	Concurrencia y punteros fuera de contrato
Integración del acumulador	I1-I3	Pruebas de integración	Acumulador correcto/defectuoso	Depende del contrato inferior
Validez del oráculo	-	Mutación, oráculos	Oráculo de especificación	Mutantes equivalentes no detectados

Invariante común a las prácticas

- Superar una batería finita de pruebas **no demuestra seguridad**.
- Cada corrección propuesta es **contextual**: válida para un modelo de amenaza y unas precondiciones declaradas, y nunca «universalmente segura».
- Lo que queda fuera de la demostración es explícito: concurrencia y validez general de punteros no forman parte del contrato y no se modelan.
- La ejecución satisfactoria de las pruebas es evidencia complementaria, no sustituto de la demostración.

Preguntas de reflexión

1. ¿Por qué un programa que pasa todas las pruebas funcionales puede seguir siendo inseguro?
2. ¿Qué parte del esfuerzo se desplaza al completar la especificación (P. 02) y qué parte *no*?
3. ¿Por qué el aislamiento es condición de las prácticas y no una formalidad?
4. ¿Qué significa que una puerta sea *fail-closed* (P. 03)? ¿Qué ocurre si falla de forma ambigua?
5. «El pipeline está verde, luego el artefacto es seguro»: ¿qué error comete?
6. ¿Qué diferencia hay entre documentar un hallazgo crítico y aceptar formalmente su riesgo residual?

¿Qué se protege exactamente con el aislamiento?

- Si las prácticas ejecutan código no fiable, ¿qué activos protege el aislamiento y qué no certifica?

Referencias I